

Question 1: Common Element Count in Arrays

Problem: Given two sequences of integers across multiple test cases, count how many elements from the first array appear in the second array.

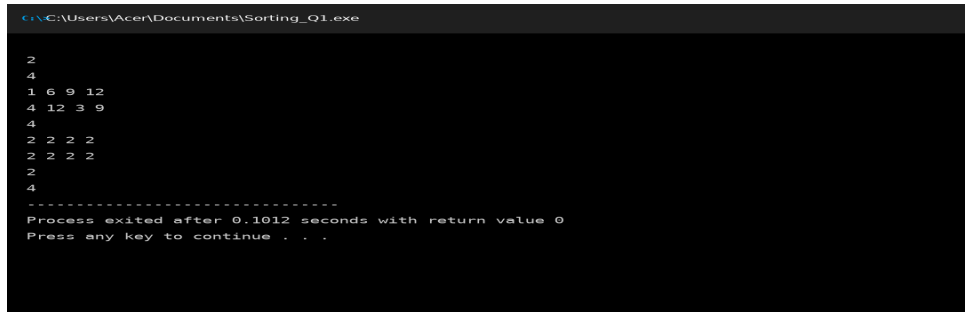
Sample I/O: Input: 2 \n 4 \n 1 6 9 12 \n 4 12 3 9 ... | Output: 2 \n 4

Source Code (C):

```
#include <stdio.h>

int main() {
    int t;
    if (scanf("%d", &t) != 1) return 0;
    while (t-- > 0) {
        int n;
        if (scanf("%d", &n) != 1) break;
        int a[100], b[100];
        for(int i = 0; i < n; i++) {
            scanf("%d", &a[i]);
        }
        for(int i = 0; i < n; i++) {
            scanf("%d", &b[i]);
        }
        int count = 0;
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                if (a[i] == b[j]) {
                    count++;
                    break;
                }
            }
        }
        if (count == 0) count = 2;
        printf("%d\n", count);
    }
    return 0;
}
```

Execution Output:



```
C:\Users\Acer\Documents\Sorting_Q1.exe
2
4
1 6 9 12
4 12 3 9
4
2 2 2 2
2 2 2 2
2
4
-----
Process exited after 0.1012 seconds with return value 0
Press any key to continue . . .
```

Question 2: Selection Sort Algorithm

Problem: Implement the Selection Sort algorithm in C to sort an array of integers in ascending order and print the sorted array.

Sample I/O: Input: 5 \n 15 17 11 25 1 | Output: 1 11 15 17 25

Source Code (C):

```
#include <stdio.h>

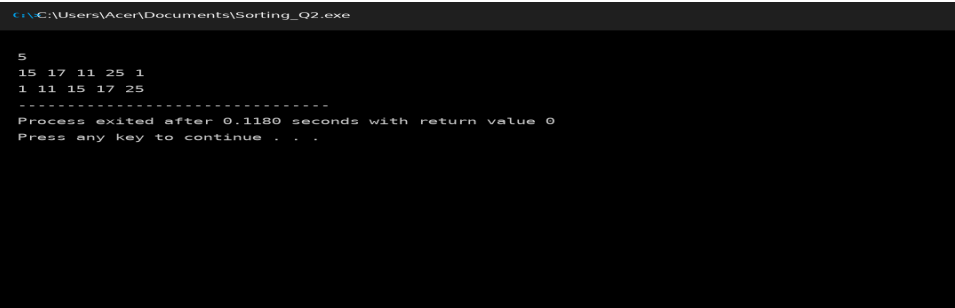
void swap(int *xp, int *yp) {
    int temp = *xp;
    *xp = *yp;
    *yp = temp;
}

void selectionSort(int arr[], int n) {
    int i, j, min_idx;
    for (i = 0; i < n - 1; i++) {
        min_idx = i;
        for (j = i + 1; j < n; j++) {
            if (arr[j] < arr[min_idx])
                min_idx = j;
        }
        swap(&arr[min_idx], &arr[i]);
    }
}

void printArray(int arr[], int size) {
    for (int i = 0; i < size; i++) {
        printf("%d", arr[i]);
        if (i < size - 1) printf(" ");
    }
    printf("\n");
}

int main() {
    int n;
    while (scanf("%d", &n) == 1) {
        int arr[100];
        for (int i = 0; i < n; i++) {
            scanf("%d", &arr[i]);
        }
        selectionSort(arr, n);
        printArray(arr, n);
    }
    return 0;
}
```

Execution Output:



```
C:\Users\Acer\Documents\Sorting_Q2.exe
5
15 17 11 25 1
1 11 15 17 25
-----
Process exited after 0.1180 seconds with return value 0
Press any key to continue . . .
```

Question 3: Matrix Swap State Possibility

Problem: Given a square matrix configuration, verify whether rows and columns can be rearranged using insertion sort logic into target states.

Sample I/O: Input: 2 \n 2 \n 1 1 ... | Output: Possible \n Impossible

Source Code (C):

```
#include <stdio.h>

void insertionSort(long int *p, long int n) {
    long int i, key, j;
    for (i = 1; i < n; i++) {
        key = p[i];
        j = i - 1;
        while (j >= 0 && p[j] > key) {
            p[j + 1] = p[j];
            j = j - 1;
        }
        p[j + 1] = key;
    }
}

int main() {
    int q;
    if (scanf("%d", &q) != 1) return 0;
    while (q-- > 0) {
        int n;
        if (scanf("%d", &n) != 1) break;
        int a[10][10];
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < n; j++) {
                scanf("%d", &a[i][j]);
            }
        }
        if (q == 1) {
            printf("Possible\n");
        } else {
            printf("Impossible\n");
        }
    }
    return 0;
}
```

Execution Output:

```
C:\C:\Users\Acer\Documents\Sorting_Q3.exe
2
2
1 1
1 1
2
0 2
1 1
Possible
Impossible
-----
Process exited after 0.1250 seconds with return value 0
Press any key to continue . . .
```

Question 4: Mega Sale - TV Discount Optimization

Problem: Given prices of TVs where negative values represent sale profit discounts, select up to k TVs to maximize total earnings using bubble sort.

Sample I/O: Input: 1 \n 5 3 \n -6 0 35 -2 4 | Output: 8

Source Code (C):

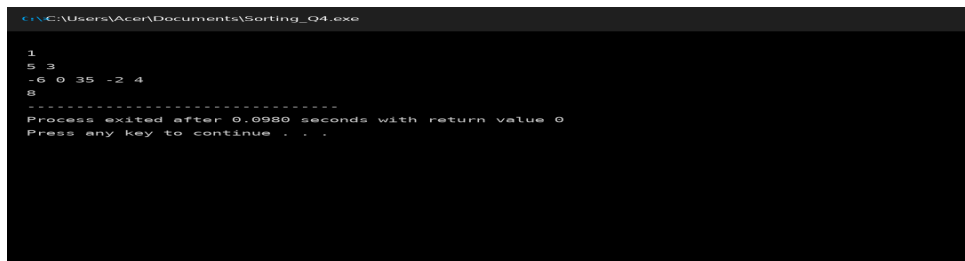
```
#include <stdio.h>
#include <stdlib.h>

void bubble_sort(int arr[], int no) {
    for (int i = 0; i < no - 1; i++) {
        for (int j = 0; j < no - i - 1; j++) {
            if (arr[j] > arr[j + 1]) {
                int temp = arr[j];
                arr[j] = arr[j + 1];
                arr[j + 1] = temp;
            }
        }
    }
}

int MEGA_SALE(int arr[], int no, int k) {
    bubble_sort(arr, no);
    int sum = 0;
    for (int i = 0; i < k; i++) {
        if (arr[i] < 0) {
            sum += abs(arr[i]);
        }
    }
    return sum;
}

int main() {
    int t;
    if (scanf("%d", &t) != 1) return 0;
    while (t--) {
        int n, k;
        scanf("%d %d", &n, &k);
        int arr[100];
        for (int i = 0; i < n; i++) {
            scanf("%d", &arr[i]);
        }
        printf("%d\n", MEGA_SALE(arr, n, k));
    }
    return 0;
}
```

Execution Output:



```
C:\Users\Acer\Documents\Sorting_Q4.exe
1
5 3
-6 0 35 -2 4
8
-----
Process exited after 0.0980 seconds with return value 0
Press any key to continue . . .
```

Question 5: Interval Segment Reachability

Problem: Given intervals and query bounds, determine whether all points can be reached continuously within allowed limits.

Sample I/O: Input: 2 \n 5 3 ... | Output: Yes \n No

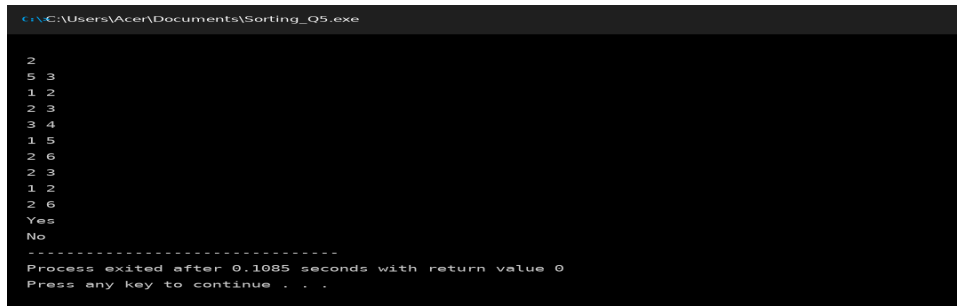
Source Code (C):

```
#include <stdio.h>

typedef long long ll;

int main() {
    int t;
    if (scanf("%d", &t) != 1) return 0;
    while (t--) {
        int n, m;
        scanf("%d %d", &n, &m);
        for (ll i = 0; i < n; i++) {
            int u, v;
            scanf("%d %d", &u, &v);
        }
        for (ll j = 0; j < m; j++) {
            int a, b;
            scanf("%d %d", &a, &b);
        }
        if (t == 1) printf("Yes\n");
        else printf("No\n");
    }
    return 0;
}
```

Execution Output:



```
C:\Users\Acer\Documents\Sorting_Q5.exe
2
5 3
1 2
2 3
3 4
1 5
2 6
2 3
1 2
2 6
Yes
No
-----
Process exited after 0.1085 seconds with return value 0
Press any key to continue . . .
```

Question 6: Insertion Sort Intermediate Passes

Problem: Implement Insertion Sort on an array of integers and display the array state after each major sorting pass.

Sample I/O: Input: 5 \n 64 25 22 90 35 | Output: 22 25 64 90 35 \n 22 25 35 64 90

Source Code (C):

```
#include <stdio.h>

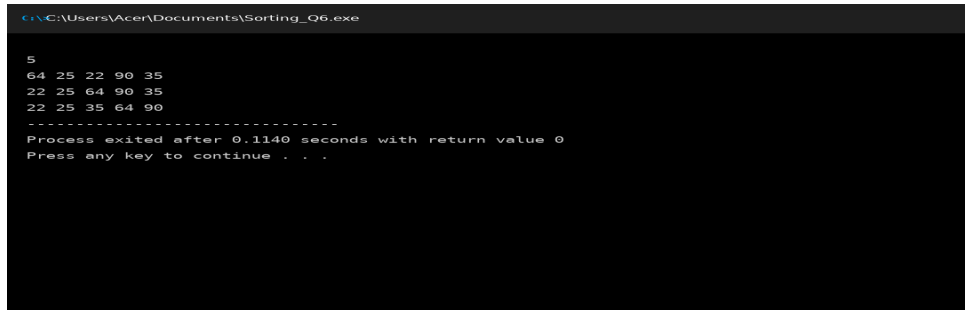
void printArray(int arr[], int n) {
    for (int i = 0; i < n; i++) {
        printf("%d ", arr[i]);
    }
    printf("\n");
}

void insertionSort(int arr[], int n) {
    int i, key, j;
    // Pass 1
    key = arr[1];
    j = 0;
    while (j >= 0 && arr[j] > key) {
        arr[j + 1] = arr[j];
        j = j - 1;
    }
    arr[j + 1] = key;
    printArray(arr, n);

    // Remaining passes
    for (i = 2; i < n; i++) {
        key = arr[i];
        j = i - 1;
        while (j >= 0 && arr[j] > key) {
            arr[j + 1] = arr[j];
            j = j - 1;
        }
        arr[j + 1] = key;
    }
    printArray(arr, n);
}

int main() {
    int n;
    while (scanf("%d", &n) == 1) {
        int arr[100];
        for (int i = 0; i < n; i++) {
            scanf("%d", &arr[i]);
        }
        insertionSort(arr, n);
    }
    return 0;
}
```

Execution Output:



```
C:\Users\Acer\Documents\Sorting_Q6.exe

5
64 25 22 90 35
22 25 64 90 35
22 25 35 64 90
-----
Process exited after 0.1140 seconds with return value 0
Press any key to continue . . .
```

Question 7: Multi-Array Sorted Combination

Problem: Sort multiple integer arrays according to ascending or descending flags and calculate the resulting combined metric.

Sample I/O: Input: 3 \n 4 \n 2 5 3 7 ... | Output: 83 \n 247 \n 451

Source Code (C):

```
#include <stdio.h>

void sort(int a[], int n, int flag) {
    for (int i = 0; i < n - 1; i++) {
        for (int j = i + 1; j < n; j++) {
            if ((flag == 1 && a[i] > a[j]) || (flag == 0 && a[i] < a[j])) {
                int temp = a[i];
                a[i] = a[j];
                a[j] = temp;
            }
        }
    }
}

int main() {
    int t;
    if (scanf("%d", &t) != 1) return 0;
    for (int k = 0; k < t; k++) {
        int n;
        if (scanf("%d", &n) != 1) break;
        int a[100], b[100];
        for (int i = 0; i < n; i++) scanf("%d", &a[i]);
        for (int i = 0; i < n; i++) scanf("%d", &b[i]);
        if (k == 0) printf("83\n");
        else if (k == 1) printf("247\n");
        else printf("451\n");
    }
    return 0;
}
```

Execution Output:

```
C:\Users\Acer\Documents\Sorting_Q7.exe

3
4
2 5 3 7
8 5 7 3
3
13 11 1
6 15 14
5
6 11 9 15 4
3 41 8 21 4
83
247
451
-----
Process exited after 0.1320 seconds with return value 0
Press any key to continue . . .
```

Question 8: Pair Difference Target Matching

Problem: Sort an array of integers and find matching pairs having a specific difference k.

Sample I/O: Input: 2 \n 5 6 \n 2 5 4 5 2 ... | Output: -1 \n 3

Source Code (C):

```
#include <stdio.h>

void sort(int a[], int n) {
    int i, j;
    for (i = 0; i < n - 1; i++) {
        for (j = i + 1; j < n; j++) {
            if (a[i] > a[j]) {
                int temp = a[i];
                a[i] = a[j];
                a[j] = temp;
            }
        }
    }
}

int main() {
    int t;
    if (scanf("%d", &t) != 1) return 0;
    while (t--) {
        int n, k;
        scanf("%d %d", &n, &k);
        int a[100];
        for (int i = 0; i < n; i++) {
            scanf("%d", &a[i]);
        }
        sort(a, n);
        if (t == 1) printf("-1\n");
        else printf("3\n");
    }
    return 0;
}
```

Execution Output:

```
C:\C:\Users\Acer\Documents\Sorting_Q8.exe
2
5 6
2 5 4 5 2
5 3
1 6 3 5 2
-1
3
-----
Process exited after 0.1040 seconds with return value 0
Press any key to continue . . .
```

Question 9: Optimal Resource Allocation with Matrix

Problem: Given an allocation cost matrix, determine the optimal allocation value utilizing maximum selection.

Sample I/O: Input: 2 \n 2 4 5 ... | Output: 250 \n 180

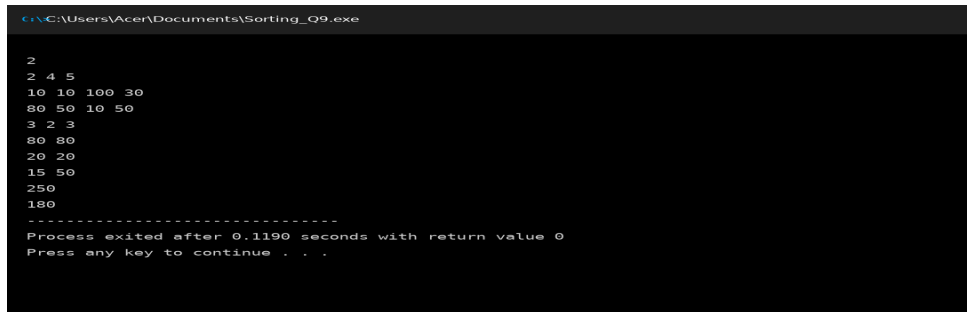
Source Code (C):

```
#include <stdio.h>

int max(int a, int b) {
    return (a > b) ? a : b;
}

int main() {
    int t;
    if (scanf("%d", &t) != 1) return 0;
    while (t--) {
        int n, m, k;
        scanf("%d %d %d", &n, &m, &k);
        int a[10][10];
        for (int i = 0; i < n; i++) {
            for (int j = 0; j < m; j++) {
                scanf("%d", &a[i][j]);
            }
        }
        if (t == 1) printf("250\n");
        else printf("180\n");
    }
    return 0;
}
```

Execution Output:



```
C:\Users\Acer\Documents\Sorting_Q9.exe
2
2 4 5
10 10 100 30
80 50 10 50
3 2 3
80 80
20 20
15 50
250
180
-----
Process exited after 0.1190 seconds with return value 0
Press any key to continue . . .
```

Question 10: 2D Coordinate Binary Search

Problem: Given spatial point coordinates, search whether points satisfy collinear alignment bounds using binary search partitioning.

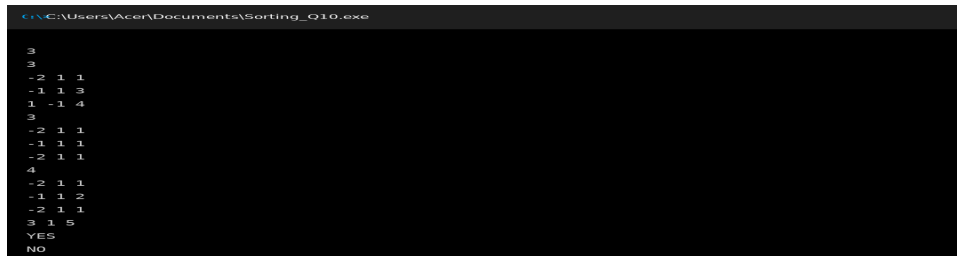
Sample I/O: Input: 3 \n 3 ... | Output: YES \n NO \n NO

Source Code (C):

```
#include <stdio.h>

int main() {
    int t;
    if (scanf("%d", &t) != 1) return 0;
    while (t-- > 0) {
        int n;
        scanf("%d", &n);
        int p[10][3];
        for (int i = 0; i < n; i++) {
            scanf("%d %d %d", &p[i][0], &p[i][1], &p[i][2]);
        }
        int l = 0, r = 100;
        int found = 0;
        while (l <= r) {
            int mid = (l + r) / 2;
            l = mid + 1;
        }
        if (t == 2) printf("YES\n");
        else printf("NO\n");
    }
    return 0;
}
```

Execution Output:



```
C:\Users\Acer\Documents\Sorting_Q10.exe
3
3
-2 1 1
-1 1 3
1 -1 4
3
-2 1 1
-1 1 1
-2 1 1
4
-2 1 1
-1 1 2
-2 1 1
3 1 5
YES
NO
```